

frank-wolfe-lean: Formal Proofs of Frank-Wolfe Convergence in Lean 4

Ben Cassie

ORCID: 0009-0004-1899-7627

DOI: 10.5281/zenodo.20478157

2026-05-31

Abstract

`frank-wolfe-lean` is a Lean 4 / Mathlib library formalising the core convergence proof for the Frank-Wolfe, or conditional gradient, method. The library works over a real inner product space, packages a compact convex feasible set, an abstract gradient map, first-order convexity and smoothness hypotheses, and a diameter bound. It proves existence of a linear minimisation oracle, feasibility of all iterates, the Frank-Wolfe gap estimates, the per-step descent inequality, and the standard $O(1/k)$ convergence rate. The development is machine-checked in Lean 4 with zero `sorry`, zero `admit`, and standard Lean/Mathlib axioms only.

1. Introduction

Frank-Wolfe is a projection-free first-order method for constrained convex optimisation. Instead of taking a gradient step and projecting back to the feasible set, it minimises a linearised objective over the feasible region and moves by a convex combination toward the resulting point. This makes the method important in settings where projection is expensive but linear optimisation over the constraint set is relatively cheap, such as simplices, spectrahedra, and norm balls.

The classical proof of Frank-Wolfe convergence combines three ingredients. First, the linear minimisation oracle gives a direction whose Frank-Wolfe gap bounds suboptimality. Second, smoothness turns the convex-combination step into a descent inequality with a quadratic curvature term. Third, the step size `gamma_k = 2 / (k + 2)` yields a recurrence that solves to the usual $O(1/k)$ rate. The repository formalises this proof spine directly, with the analytic side conditions recorded in a single `FrankWolfeSetup` structure.

The result is not a new optimisation theorem. It is a reusable Lean 4 formalisation of the standard projection-free convergence argument under explicit compactness, convexity, smoothness, and diameter hypotheses.

2. Library Overview

The project is organised into four implementation modules plus a root import file:

- `FrankWolfe/Defs.lean` defines `FrankWolfeSetup`, the step size, the Frank-Wolfe step, the linear minimisation oracle, and the iterate sequence.

- `FrankWolfe/FWGap.lean` defines the Frank-Wolfe gap and proves non-negativity, suboptimality domination, and the optimality certificate.
- `FrankWolfe/Descent.lean` proves displacement identities, the step norm bound, and the per-step descent lemma.
- `FrankWolfe/Convergence.lean` proves the suboptimality recurrence, a gap-sum bound, the first-iterate bound, and the final convergence theorem.
- `FrankWolfe.lean` is the root module importing the library.

The project depends on:

- Lean v4.28.0
- Mathlib v4.28.0

The formal development contains zero `sorry`, zero `admit`, and introduces no project-specific axioms. It is written against Lean 4 and Mathlib, using standard Lean/Mathlib axioms only.

The formal setting is a complete real inner product space:

```
variable {E : Type*} [NormedAddCommGroup E]
      [InnerProductSpace Real E] [CompleteSpace E]
```

The setup structure contains a feasible set C , an objective f , a gradient map `grad_f`, smoothness constant L , diameter bound D , compactness and convexity of C , and the first-order hypotheses

```
f x + <grad_f x, y - x> <= f y
f y <= f x + <grad_f x, y - x> + L / 2 * ||y - x||^2
||x - y|| <= D
```

for feasible x and y .

The repository is available at:

<https://github.com/velvetmonkey/frank-wolfe-lean>

3. Theorem Inventory

The source contains sixteen named theorem-level results, organised into definitions and feasibility, gap facts, per-step analysis, and convergence.

Layer 1 - Definitions and Feasibility

1. `stepSize_pos` proves that the Frank-Wolfe step size is positive:

`0 < stepSize k`.

2. `stepSize_le_one` proves that the step size is at most one:

`stepSize k <= 1`.

3. `one_sub_stepSize_nonneg` proves the corresponding convex-combination coefficient is non-negative:

$0 \leq 1 - \text{stepSize } k$.
 4. `lmo_exists` proves existence of a linear minimisation oracle response over the compact feasible set:
`exists s, s in C and forall y in C, <g, s> <= <g, y>.`
 5. `linearMinimisationOracle_spec` states that the chosen noncomputable LMO response satisfies the specification.
 6. `lmo_mem` proves that the LMO response belongs to `C`.
 7. `lmo_le` proves the linear-minimisation inequality for the chosen response.
 8. `fwStep_mem` proves feasibility of one Frank-Wolfe step:
`x in C, s in C, 0 <= gamma <= 1`
`-> fwStep x s gamma in C.`
 9. `fwIterates_zero` and `fwIterates_succ` unfold the iterate sequence at zero and successor steps.
 10. `fwIterates_mem` proves that every iterate remains feasible when `x0 in C`.

Layer 2 - Frank-Wolfe Gap

11. `fw_gap_nonneg` proves non-negativity of the Frank-Wolfe gap:
 $0 \leq \langle \text{grad}_f x, x - s \rangle$.
 12. `fwGapAt_nonneg` lifts gap non-negativity to every iterate.
 13. `fw_gap_bounds_subopt` proves that the gap upper-bounds suboptimality against any feasible point:
 $f x - f y \leq \text{fwGap } x \ s$.
 14. `fw_gap_optimality` proves that a zero gap gives global optimality over `C`.

Layer 3 - Per-Step Analysis

15. `fw_step_sub` rewrites the Frank-Wolfe displacement:
 $\text{fwStep } x \ s \ \text{gamma} - x = \text{gamma} * (s - x)$.
 16. `inner_grad_fw_step` rewrites the smoothness inner product in terms of the gap:
 $\langle \text{grad}_f x, \text{fwStep } x \ s \ \text{gamma} - x \rangle = -(\text{gamma} * \text{fwGap } x \ s)$.
 17. `fw_step_norm_sq_le` bounds the squared step length by the diameter:
 $\|\text{fwStep } x \ s \ \text{gamma} - x\|^2 \leq \text{gamma}^2 * D^2$.
 18. `fw_descent_lemma` proves the local descent estimate:

$f(\text{fwStep } x \ s \ \text{gamma})$
 $\leq f \ x - \text{gamma} * \text{fwGap } x \ s + \text{gamma}^2 * (L / 2) * D^2.$

Layer 4 - Convergence

19. `fw_subopt_recurrence` gives the one-step suboptimality recurrence:

$h_{\{k+1\}} \leq (1 - \text{gamma}_k) * h_k + \text{gamma}_k^2 * (L / 2) * D^2.$

20. `fw_gap_sum_bound` telescopes the descent lemma into a finite gap-sum bound.

21. `fw_first_iterate_bound` proves the base estimate after the first step:

$f(x_1) - f(x^*) \leq L / 2 * D^2.$

22. `nat_ineq` supplies the arithmetic inequality used in the induction.

23. `frank_wolfe_convergence` is the main theorem:

$f(x_{\{k+1\}}) - f(x^*) \leq 2 * L * D^2 / (k + 3).$

4. Main Theorems

Linear Minimisation Oracle

The theorem `lmo_exists` uses compactness of $\mathcal{C}$ and continuity of the linear functional $y \mapsto \langle g, y \rangle$ to prove that the oracle exists. The selected response is noncomputable, but its specification is available through `linearMinimisationOracle_spec`. This is the formal replacement for the informal algorithmic instruction “call the LMO”.

Gap Bounds Suboptimality

The theorem `fw_gap_bounds_subopt` is the bridge between the oracle and optimisation error. Convexity gives

$f \ x - f \ y \leq \langle \text{grad}_f \ x, x - y \rangle.$

The LMO inequality gives

$\langle \text{grad}_f \ x, s \rangle \leq \langle \text{grad}_f \ x, y \rangle.$

Together these imply

$f \ x - f \ y \leq \langle \text{grad}_f \ x, x - s \rangle.$

Descent Lemma

The theorem `fw_descent_lemma` applies the smoothness upper bound to `fwStep x s gamma`. The displacement identity converts the linear term to `-gamma * fwGap x s`, and the diameter bound controls the quadratic term. This gives the one-step inequality from which the global rate follows.

Convergence Rate

The theorem `frank_wolfe_convergence` assumes an initial feasible point `x0`, an optimal feasible point `xstar`, and optimality of `xstar` over `C`. It proves

$$\begin{aligned} &P.f \ (P.fwIterates \ x0 \ (k + 1)) - P.f \ xstar \\ &\leq 2 * P.L * P.D^2 / (k + 3). \end{aligned}$$

This is the standard $O(1/k)$ Frank-Wolfe convergence rate in the indexed form used by the library.

5. Proof Sketch

The proof begins in `FrankWolfe/Defs.lean`. Compactness gives an LMO response for each gradient direction. Convexity of `C` and the inequalities $0 < \gamma_k$ and $\gamma_k \leq 1$ show that each Frank-Wolfe step remains feasible. Induction over the recursive iterate definition then proves `fwIterates_mem`.

`FrankWolfe/FWGap.lean` proves that the LMO makes the gap non-negative and large enough to dominate suboptimality. The zero-gap optimality theorem is a direct consequence of the same gap bound.

`FrankWolfe/Descent.lean` performs the one-step calculation. It rewrites the update as a displacement, rewrites the inner product against `grad_f x` as minus the gap, and bounds the squared displacement by $\gamma^2 D^2$. Substituting these facts into the smoothness hypothesis yields `fw_descent_lemma`.

`FrankWolfe/Convergence.lean` combines the descent lemma with the gap suboptimality bound to obtain the recurrence. The base case follows from $\gamma_0 = 1$, and the final induction uses arithmetic on $\gamma_k = 2 / (k + 2)$ to derive the displayed $O(1/k)$ bound.

6. Relation to Sibling Libraries

`frank-wolfe-lean` sits in the same optimisation suite as `gradient-descent-lean`, `projected-gd-lean`, `sgd-lean`, and `mirror-descent-lean`. `gradient-descent-lean` has DOI 10.5281/zenodo.20472996 and proves smooth convex gradient descent convergence. `projected-gd-lean` has DOI 10.5281/zenodo.20475662 and adds a projection operator for constrained optimisation. Frank-Wolfe addresses the same constrained setting from a projection-free angle by using a linear minimisation oracle.

`mirror-descent-lean` has DOI 10.5281/zenodo.20475033 and treats constrained optimisation through Bregman projection optimality. `sgd-lean` has DOI 10.5281/zenodo.20475583 and proves a bounded-noise stochastic-gradient surrogate. These libraries share a proof pattern: local inequality, telescoping or recurrence, and explicit rate. The present library contributes the conditional-gradient member of that family.

7. Conclusion

`frank-wolfe-lean` provides a compact Lean 4 / Mathlib formalisation of Frank-Wolfe convergence. It defines the feasible optimisation setup, proves existence and correctness facts for the linear minimisation oracle, proves feasibility of iterates, establishes the Frank-Wolfe gap properties, and proves the descent and recurrence estimates needed for the final $O(1/k)$ theorem.

Future work could instantiate the abstract feasible set with concrete domains such as simplices or spectrahedra, formalise curvature constants beyond the diameter-based smoothness bound, and connect the LMO specification to executable optimisation routines. The present repository supplies the reusable convergence spine.

References

- Frank, M. and Wolfe, P. (1956). *An algorithm for quadratic programming*. Naval Research Logistics Quarterly, 3(1-2), 95-110.
- Jaggi, M. (2013). *Revisiting Frank-Wolfe: Projection-free sparse convex optimization*. Proceedings of the 30th International Conference on Machine Learning.
- The Mathlib Community. (2024). *The Lean Mathematical Library*. GitHub repository. <https://github.com/leanprover-community/mathlib4>
- Cassie, B. (2026). *gradient-descent-lean: Formal Proofs of Gradient Descent Convergence in Lean 4*. Zenodo. <https://doi.org/10.5281/zenodo.20472996>
- Cassie, B. (2026). *projected-gd-lean: Formal Proofs of Projected Gradient Descent in Lean 4*. Zenodo. <https://doi.org/10.5281/zenodo.20475662>
- Cassie, B. (2026). *sgd-lean: Formal Proofs of Bounded-Noise SGD Convergence in Lean 4*. Zenodo. <https://doi.org/10.5281/zenodo.20475583>
- Cassie, B. (2026). *mirror-descent-lean: Formal Proofs of Mirror Descent and Bregman Divergence Convergence in Lean 4*. Zenodo. <https://doi.org/10.5281/zenodo.20475033>